

Khai thác tập phổ biến

Phân tích thuật toán FP-Growth và ứng dụng

Nguyễn Hữu Khánh Hưng Phạm Quốc Khánh Y Nguyên Miô
Vũ Trần Phúc Trần Nguyễn Minh Quân

Trường Đại học Khoa học Tự nhiên – ĐHQG-HCM
Khoa Công nghệ Thông tin – Môn học: Khai thác dữ liệu và ứng dụng

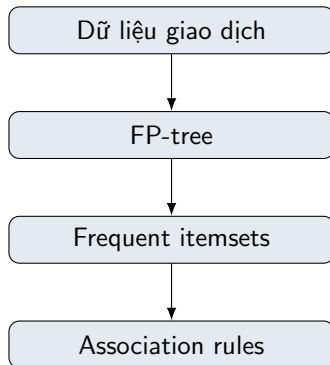
Học kỳ 2 – 2025–2026

- ① Bối cảnh và nền tảng
- ② Thuật toán FP-Growth
- ③ Cài đặt và tối ưu hóa
- ④ Thực nghiệm và đánh giá
- ⑤ Ứng dụng Market Basket Analysis
- ⑥ Kết luận

Bài toán trung tâm

Khai thác tất cả tập mục phổ biến trong cơ sở dữ liệu giao dịch, sau đó dùng kết quả để sinh luật kết hợp có ý nghĩa.

- Trình bày nền tảng toán học của frequent itemset mining.
- Phân tích FP-Growth: FP-tree, conditional pattern-base, pattern growth.
- Cài đặt hai phiên bản bằng Julia: cơ bản và tối ưu hóa.
- Đánh giá trên benchmark và ứng dụng Market Basket Analysis.



Cơ sở dữ liệu giao dịch

$$D = \{T_1, T_2, \dots, T_n\}, \quad T_k \subseteq I$$

Một itemset X xuất hiện trong giao dịch T_k khi $X \subseteq T_k$.

Support

$$\text{sup}(X) = |\{T \in D \mid X \subseteq T\}|,$$
$$\text{rsup}(X) = \frac{\text{sup}(X)}{|D|}.$$

Frequent itemset

Với ngưỡng minimum support ξ :

$$X \text{ phổ biến} \iff \text{sup}(X) \geq \xi$$

- **Đầu vào:** cơ sở dữ liệu D , ngưỡng ξ .
- **Đầu ra:** tập đầy đủ F gồm mọi itemset phổ biến và support tương ứng.

Bảng giao dịch

TID	Các mặt hàng
T1	{Sữa, Bánh mì}
T2	{Sữa, Trứng}
T3	{Sữa, Bánh mì}
T4	{Bánh mì, Trứng}

Support

- $\text{sup}(\{\text{Sữa}\}) = 3$
- $\text{sup}(\{\text{Sữa, Bánh mì}\}) = 2$
- $\text{rsup}(\{\text{Sữa, Bánh mì}\}) = 2/4 = 50\%$

Ngưỡng phổ biến

- Chọn $\xi = 2$.
- {Sữa}: phổ biến ($3 \geq 2$).
- {Bánh mì}: phổ biến ($3 \geq 2$).
- {Trứng}: phổ biến ($2 \geq 2$).
- {Sữa, Bánh mì}: phổ biến ($2 \geq 2$).
- {Sữa, Trứng}: không phổ biến ($1 < 2$).

Kết luận

Đầu vào: D và $\xi = 2 \rightarrow$ Đầu ra: $F = \{\{\text{Sữa}\}, \{\text{Bánh mì}\}, \{\text{Trứng}\}, \{\text{Sữa, Bánh mì}\}\}$ cùng support tương ứng.

Quan hệ bao hàm

$$M \subseteq C \subseteq F$$

- **Closed itemset:** không có tập cha phổ biến nào có cùng support.
- **Maximal itemset:** không có tập cha phổ biến nào.

Anti-monotone property

$$X \subseteq Y \Rightarrow \text{sup}(X) \geq \text{sup}(Y)$$

- Nếu X phổ biến thì mọi tập con của X cũng phổ biến.
- Nếu X không phổ biến thì mọi tập cha của X đều bị loại.
- Đây là nền tảng cốt lõi không gian tìm kiếm.

Bảng giao dịch

TID	Các mặt hàng
T1	{Sữa, Bánh mì}
T2	{Sữa, Trứng}
T3	{Sữa, Bánh mì}
T4	{Bánh mì, Trứng}

Support

- $\text{sup}(\{\text{Sữa, Trứng}\}) = 1$
- $\text{sup}(\{\text{Sữa, Bánh mì, Trứng}\}) = 0$

Kết luận

Nếu một itemset không phổ biến thì mọi tập cha của nó cũng không phổ biến $\rightarrow$ cắt bỏ cả nhánh tìm kiếm.

Hệ quả anti-monotone

- Với $\xi = 2$: {Sữa, Trứng} không phổ biến ($1 < 2$).
- Mọi tập cha của {Sữa, Trứng} chắc chắn cũng không phổ biến.
- {Sữa, Bánh mì, Trứng} $\supseteq$ {Sữa, Trứng} $\Rightarrow$ bị loại ngay, không cần đếm.

Thuật toán FP-Growth

Vì sao FP-Growth?

Hạn chế của Apriori

- Sinh candidate itemset theo từng mức k .
- Phải quét cơ sở dữ liệu nhiều lần.
- Số ứng viên bùng nổ khi dữ liệu dày hoặc itemset dài.

Ý tưởng FP-Growth

- Không sinh candidate itemset.
- Nén cơ sở dữ liệu vào FP-tree sau đúng hai lần quét.
- Khai thác bằng pattern growth và chia để trị.

Thông điệp chính

FP-Growth thay chi phí sinh ứng viên bằng chi phí xây và khai thác cấu trúc cây nén, thường hiệu quả hơn rõ rệt trên cơ sở dữ liệu giao dịch lớn.

Thuật toán FP-Growth

FP-tree: cấu trúc dữ liệu cốt lõi

Mỗi nút gồm

- **item-name:** tên item.
- **count:** số giao dịch đi qua đường dẫn đến nút.
- **node-link:** liên kết đến nút cùng item trong cây.

Header table

Lưu item phổ biến và con trỏ đến chuỗi node-link của item đó.

Nguyên tắc nén

- ① Chỉ giữ item phổ biến.
- ② Sắp xếp theo support giảm dần.
- ③ Các giao dịch cùng tiền tố dùng chung đường đi.

Tính chất

FP-tree đầy đủ nhưng thường nhỏ hơn dữ liệu gốc nhờ chia sẻ tiền tố.

Ví dụ minh họa chia sẻ tiền tố trên FP-tree

Hai giao dịch mẫu

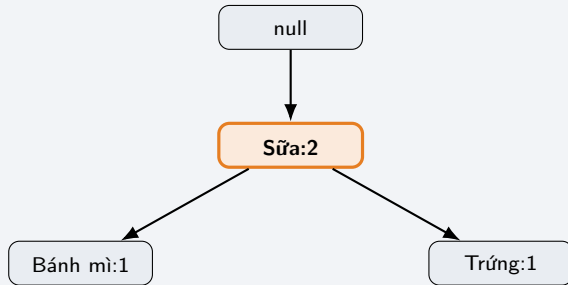
Đã sắp theo support giảm dần:

- T1 = ⟨Sữa, Bánh mì⟩
- T2 = ⟨Sữa, Trứng⟩

Các bước chèn

- Chèn T1: tạo đường **Sữa** → Bánh mì, mỗi nút đếm 1.
- Chèn T2: dùng chung nút **Sữa**, tăng bộ đếm thành 2; rẽ nhánh mới Trứng.

Cây sau khi chèn T1 và T2



Kết luận

Giao dịch chung tiền tố dùng chung đường đi; nút chia sẻ chỉ tăng bộ đếm → cây nén nhưng không mất thông tin.

Xây dựng FP-tree

- 1 Quét lần 1: đếm support item đơn.
- 2 Lọc theo ξ , tạo FList giảm dần theo support.
- 3 Quét lần 2: sắp xếp từng giao dịch theo FList.
- 4 Chèn giao dịch vào cây, tăng count khi chia sẻ tiền tố.

Khai thác mẫu

- 1 Duyệt header table từ support thấp đến cao.
- 2 Tạo conditional pattern-base cho từng item.
- 3 Xây conditional FP-tree.
- 4 Đệ quy để tăng trưởng mẫu.

Trường hợp đường đơn

Nếu FP-tree chỉ có một đường, liệt kê mọi tổ hợp con của đường đó thay vì đệ quy tiếp.

Khi nào áp dụng?

Sau khi xây conditional FP-tree cho hậu tố α , nếu cây chỉ còn một đường:

$$P = \langle (i_1, c_1), (i_2, c_2), \dots, (i_m, c_m) \rangle$$

thì dừng đệ quy và không tạo thêm conditional pattern-base.

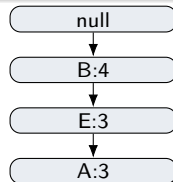
Quy tắc sinh mẫu trực tiếp

Với mọi tập con không rỗng $S \subseteq \{i_1, \dots, i_m\}$, xuất:

$$\alpha \cup S, \quad \sup(\alpha \cup S) = \min_{i_j \in S} c_j.$$

Ý nghĩa pruning

- Cắt toàn bộ nhánh đệ quy phía dưới đường đơn.
- Thay nhiều lần xây cây bằng phép liệt kê tổ hợp.
- Số mẫu sinh ra là $2^m - 1$, nhưng không quét dữ liệu thêm.



Ví dụ $\alpha = \{D\}$: sinh các mẫu với D ngay từ đường $B \rightarrow E \rightarrow A$.

Thuật toán FP-Growth

Ví dụ 1: Cơ sở dữ liệu và quét lần 1

Cơ sở dữ liệu gốc

TID	Items
T1	{A, B, D, E}
T2	{B, C, E}
T3	{A, B, D, E}
T4	{A, B, C, E}
T5	{A, B, C, D, E}
T6	{B, C, D}

Quét lần 1: Đếm support

A	B	C	D	E
4	6	4	4	5

- Ngưỡng $\xi = 3$.
- Tất cả 5 items đều phổ biến.
- Sắp xếp theo support giảm dần:
 $FList = \langle B : 6, E : 5, A : 4, C : 4, D : 4 \rangle$.

Thuật toán FP-Growth

Ví dụ 1: Sắp xếp giao dịch theo FList

Giao dịch gốc

TID	Items
T1	{A, B, D, E}
T2	{B, C, E}
T3	{A, B, D, E}
T4	{A, B, C, E}
T5	{A, B, C, D, E}
T6	{B, C, D}

Sau khi sắp xếp theo FList

TID	Đã sắp xếp
T1	$\langle B, E, A, D \rangle$
T2	$\langle B, E, C \rangle$
T3	$\langle B, E, A, D \rangle$
T4	$\langle B, E, A, C \rangle$
T5	$\langle B, E, A, C, D \rangle$
T6	$\langle B, C, D \rangle$

Mỗi giao dịch chỉ giữ items phổ biến, sắp theo thứ tự FList để tăng chia sẻ tiền tố.

Thuật toán FP-Growth

Ví dụ 1: Xây dựng FP-tree (chèn T1–T3)

- **Chèn T1** = $\langle B, E, A, D \rangle$:

Tạo đường mới: $B:1 \rightarrow E:1 \rightarrow A:1 \rightarrow D:1$.

- **Chèn T2** = $\langle B, E, C \rangle$:

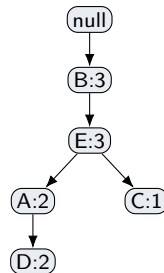
Chia sẻ tiền tố $\langle B, E \rangle$, tăng count.

Tạo nút mới $C:1$ là con của $E:2$.

- **Chèn T3** = $\langle B, E, A, D \rangle$:

Trùng hoàn toàn với đường T1.

Tăng toàn bộ: $B:3, E:3, A:2, D:2$.



FP-tree sau T1, T2, T3.

Thuật toán FP-Growth

Ví dụ 1: Xây dựng FP-tree (chèn T4–T6)

- **Chèn T4** = $\langle B, E, A, C \rangle$:

Chia sẻ $\langle B, E, A \rangle$. Tạo $C:1$ con của $A:3$.

- **Chèn T5** = $\langle B, E, A, C, D \rangle$:

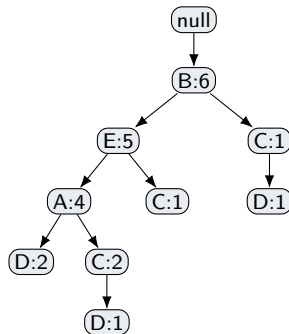
Chia sẻ $\langle B, E, A, C \rangle$ (nút C con của A).

Tạo $D:1$ con của $C:2$.

- **Chèn T6** = $\langle B, C, D \rangle$:

Chia sẻ chỉ nút B. Tạo nhánh mới:

$C:1 \rightarrow D:1$ con của $B:6$.



FP-tree hoàn chỉnh.

Thuật toán FP-Growth

Ví dụ 1: Khai thác item D

Conditional pattern-base của D

3 nút D trong FP-tree:

- $D:2$: tiền tố $\langle B:2, E:2, A:2 \rangle$
- $D:1$: tiền tố $\langle B:1, E:1, A:1, C:1 \rangle$
- $D:1$: tiền tố $\langle B:1, C:1 \rangle$

Xây conditional FP-tree | D

Đếm: $B=4, E=3, A=3, C=2$ (loại).

Kết quả: đường đơn $B:4 \rightarrow E:3 \rightarrow A:3$.

Frequent itemset chứa D

Tổ hợp	Itemset (sup)
—	$\{D\}$ (4)
$\{B\}$	$\{B,D\}$ (4)
$\{E\}$	$\{E,D\}$ (3)
$\{A\}$	$\{A,D\}$ (3)
$\{B,E\}$	$\{B,E,D\}$ (3)
$\{B,A\}$	$\{B,A,D\}$ (3)
$\{E,A\}$	$\{E,A,D\}$ (3)
$\{B,E,A\}$	$\{B,E,A,D\}$ (3)

Thuật toán FP-Growth

Ví dụ 1: Khai thác item C, A, E, B

Item C (sup=4)

- CPB: {(BEA:2), (BE:1), (B:1)}
- Đếm: B=4, E=3, A=2 (loại)
- Cond. FP-tree: $B:4 \rightarrow E:3$
- Kết quả: {C, BC, EC, BEC}

Item A (sup=4)

- CPB: {(BE:4)}
- Cond. FP-tree: $B:4 \rightarrow E:4$
- Kết quả: {A, BA, EA, BEA}

Item E (sup=5)

- CPB: {(B:5)}
- Cond. FP-tree: $B:5$
- Kết quả: {E, BE}

Item B (sup=6)

- CPB: rỗng (B là con trực tiếp gốc)
- Kết quả: {B}

Tổng kết

19 frequent itemset

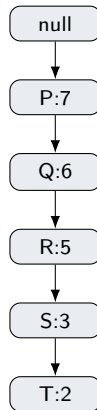
Thuật toán FP-Growth

Ví dụ 2: Tình huống đặc biệt — Dữ liệu và FP-tree

Cơ sở dữ liệu ($\xi = 2$)

TID	Items
T1	{P, Q, R, S, T}
T2	{P, Q, R, S, T}
T3	{P, Q, R, S}
T4	{P, Q, R}
T5	{P, Q, R}
T6	{P, Q}
T7	{P}

Thiết kế: mỗi giao dịch là tiền tố của giao dịch dài nhất.



FP-tree chỉ gồm **một đường đơn duy nhất** — không có nút phân nhánh.

Không tối ưu (đệ quy thông thường)

- Khai thác T: xây cond. FP-tree gồm $\langle P, Q, R, S \rangle$ — lại đường đơn.
- Khai thác S: xây cond. FP-tree gồm $\langle P, Q, R \rangle$ — lại đường đơn.
- Tiếp tục đệ quy...

Có tối ưu (Lemma 3.2)

- Nhận diện đường đơn ngay.
- Liệt kê tất cả $2^5 - 1 = 31$ tổ hợp con.
- Support = min(count các nút trong tổ hợp).
- **Không cần xây bất kỳ conditional FP-tree nào.**

Ví dụ: $\{P, R, S\}$ có sup = $\min(7, 5, 3) = 3$.

Cấu trúc Repository

- `algorithm/`: Chứa bản Base (chuẩn lý thuyết) và Optimized (hiệu năng cao).
- `io/` & `test/`: Quản lý dữ liệu định dạng SPMF và tự động benchmark.
- **Đối chiếu (Baseline)**: Sử dụng SPMF (Java) để kiểm chứng correctness và so sánh.

4 bottleneck → 4 Kỹ thuật Tối ưu

- ① Độ quy sâu vô ích → **Single Path Pruning**.
- ② Lọc item tổn kém → **BitArray Filtering**.
- ③ Phân mảnh bộ nhớ → **Buffer Reuse**.
- ④ Thắt thoát do ép kiểu động → **Type Stability & Inlining**.

1. Single Path Pruning

Nhận diện conditional FP-tree chỉ có một nhánh thẳng. Thay vì xây cây con, **sinh trực tiếp** toàn bộ $2^k - 1$ tổ hợp tập phổ biến bằng bitmask.



⇒ Dùng toán học sinh trực tiếp 31 luật (VD: $\{P, T\} : 2$), **giảm 15 lần** đệ quy xây cây vô ích.

2. BitArray Filtering

Lần quét dữ liệu gốc: Dùng mảng bit để đánh dấu item đạt MinSup thay vì tra cứu Hash Map liên tục.

- $\text{mask}[\text{id}] = \text{count} \geq x_i$
- Truy cập $O(1)$ ở mức phần cứng.
- Tăng tốc độ khởi tạo Initial Tree triệt để.

Cài đặt và tối ưu hóa

Tối ưu Bộ nhớ: Buffer Reuse (Tái sử dụng bộ đệm)

Từ Vấn đề đến Giải pháp

Vấn đề phân mảnh RAM:

Mỗi nhánh đệ quy tạo vector tạm để chứa giao dịch. Nếu cấp phát liên tục, Garbage Collector (GC) sẽ quá tải, gây giật lag.

Giải pháp Zero Allocation:

Cấp phát sẵn bộ đệm dùng xuyên suốt cây đệ quy:

- `path_buf`: tái sử dụng liên tục.
- `item_counts_stack`: Hash Map cục bộ.

Luồng thực thi với Buffer

- ❶ **Làm sạch nhưng giữ capacity:**
`empty!(path_buf)`
- ❷ **Ghi đè trực tiếp:**
Lọc và nạp transaction mới trực tiếp đè lên `path_buf`.
- ❸ **Tiêu thụ ngay lập tức:**
Đẩy `path_buf` vào cây, không lưu giữ các mảng clone (alias).

3. Type Stability

Loại bỏ hoàn toàn kiểu chung chung như Any.
Tối ưu thông qua **Parametric Types**.

- `FPNode{T}` cho phép cấu trúc Node định dạng theo số nguyên một cách linh hoạt.
- Xóa sổ "Dynamic Dispatch" tồn kém khi Julia không phải phán đoán kiểu dữ liệu lúc runtime.

4. Inlining

Xử lý các đoạn mã lỗi bị gọi hàng triệu lần
(`find_child`, `_insert_tree`).

- Dùng macro `@inline` yêu cầu compiler dán nguyên mã lệnh vào vòng lặp → cắt bỏ chi phí "nhảy dòng" (jump overhead).
- Giúp code Julia chạy với vận tốc gần ngang bằng ngôn ngữ C/C++.

Đánh giá Hiệu năng (Dataset: Connect-4, MinSup 75%)

So sánh Base và Optimized cho thấy cải thiện thời gian, nhưng có đánh đổi bộ nhớ khi output vượt 1,5 triệu itemsets.

Tốc độ thực thi

Nhanh hơn 6,6%

2,386s → 2,228s

Sử dụng Bộ nhớ (RAM)

Tăng 43,0%

584,10MB → 835,15MB

Tối ưu hóa cần được đánh giá đồng thời theo thời gian và bộ nhớ, đặc biệt trên dữ liệu dày với MinSup thấp.

Thực nghiệm và đánh giá

Mục tiêu và phương pháp đo

Thiết lập chung

Mục tiêu

Correctness, runtime, peak memory và khả năng mở rộng.

Công cụ

Julia 1.12.6; SPMF 2.42; Microsoft OpenJDK 21; Windows 11.

So sánh

Base, Optimized và SPMF trên cùng dữ liệu benchmark, MinSup và định dạng đầu ra.

Đo thời gian

- Warm-up JIT trước khi ghi nhận thời gian Julia.
- Base và Optimized chạy 5 lần.
- Báo cáo **trung vị**; SPMF chạy 1 lần theo notebook.

Đo bộ nhớ

- Julia: peak RSS của worker sau khi trừ baseline.
- SPMF: Max memory usage do SPMF báo cáo.
- Hai nền tảng được báo cáo riêng, không trộn cách đo.

Nguyên tắc công bằng: cố định dữ liệu, MinSup và định dạng đầu ra; báo cáo riêng cách đo bộ nhớ theo từng nền tảng.

Thực nghiệm và đánh giá

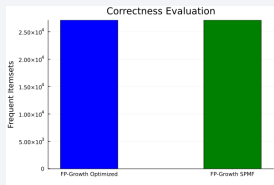
Tập dữ liệu benchmark

Dataset	Giao dịch	Items	Avg. length	Loại	Vai trò đánh giá
Connect-4	67.557	126	42,0	Dày	Bùng nổ itemset, đường đơn dài.
T10I4D100K	100.000	~870	10,0	Thưa	Bán lẻ tổng hợp, đầu ra kiểm soát.
T20I6D100K	99.910	893	20,0	Thưa	Ảnh hưởng độ dài giao dịch.
Accidents	340.183	468	33,8	Dày	Quy mô lớn, áp lực bộ nhớ.
Retail	88.162	16.470	10,3	Thưa	Dữ liệu bán lẻ thực tế.

Thực nghiệm và đánh giá

Correctness: đối chiếu với SPMF

Kiểm chứng tự động



- Unit tests: 105/105, gồm correctness, reader và smoke tests.
- Benchmark: 5/5 cấu hình đại diện khớp SPMF 100%.

Kết quả benchmark

Dataset	MinSup	Itemsets	Match
Connect-4	90%	27.127	100%
T10I4D100K	5%	10	100%
T20I6D100K	5%	99	100%
Accidents	50%	8.057	100%
Retail	50%	1	100%

Match rate 100% trên cả dữ liệu dày, thưa và quy mô lớn.

Kết luận

Khớp 100% với SPMF tại các ngưỡng đại diện, nên phần đánh giá hiệu năng có cùng cơ sở correctness.

Thực nghiệm và đánh giá

Hiệu quả tối ưu hóa: Base vs Optimized

Runtime

Cấu hình	Base	Opt.	Kết luận
Connect-4 75%	2,386	2,228	nhanh hơn 6,6%
Connect-4 80%	0,653	0,530	nhanh hơn 18,8%
Accidents 30%	3,262	3,675	chậm hơn 12,7%
Accidents 40%	2,318	1,754	nhanh hơn 24,3%
Retail 30%	0,025	0,020	nhanh hơn ~18%

Đơn vị: giây; báo cáo trung vị của 5 lần chạy.

Diễn giải

- Nhanh hơn ở 4/5 cấu hình minh họa.
- Mức cải thiện phụ thuộc dataset và MinSup.
- Accidents 30% chậm hơn do overhead tối ưu.

Runtime cần đọc cùng RAM.

Thông điệp

Tối ưu hóa giảm thời gian trong đa số cấu hình, nhưng vẫn cần kiểm tra ngoại lệ theo từng ngưỡng.

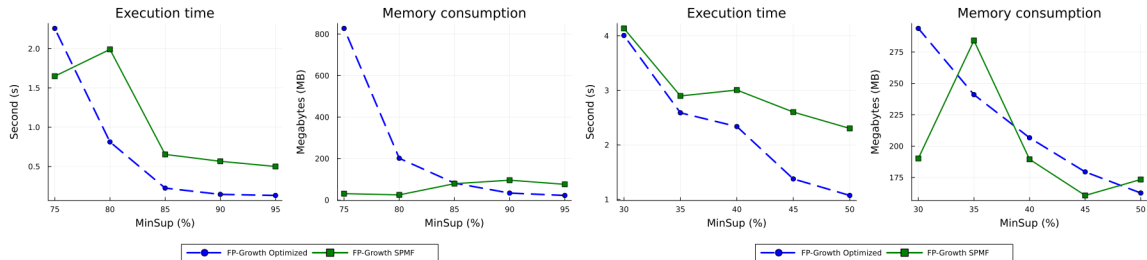
Thực nghiệm và đánh giá

Thời gian chạy: Optimized so với SPMF

Dataset	MinSup	Itemsets	Optimized (s)	SPMF (s)	Nhận xét
Connect-4	90%	27.127	0,145	0,566	Julia nhanh hơn ở ngưỡng trung bình/cao.
Connect-4	75%	1.585.551	2,258	1,650	SPMF lợi thế khi đầu ra cực lớn.
T10I4D100K	1%	385	0,440	0,967	Julia nhanh hơn trên dữ liệu thưa.
T20I6D100K	1%	1.523	2,329	4,656	Vẫn giữ lợi thế khi giao dịch dài hơn.
Accidents	50%	8.057	1,074	2,302	Nhanh hơn trên dữ liệu dày quy mô lớn.
Retail	50%	1	0,019	0,320	Nhanh hơn rõ do dữ liệu rất thưa.

Thực nghiệm và đánh giá

Hiệu năng trên dữ liệu dày



Connect-4: output bùng nổ mạnh khi MinSup giảm.

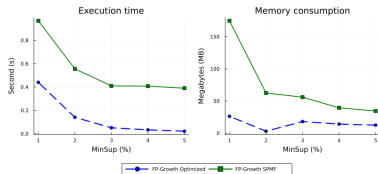
Accidents: dữ liệu lớn và dày, tạo áp lực thời gian và bộ nhớ.

Nhận xét

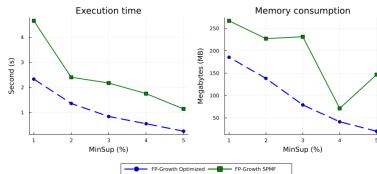
Optimized nhanh hơn SPMF ở phần lớn ngưỡng khảo sát. Riêng Connect-4 tại MinSup 75%, hơn 1,58 triệu itemsets khiến SPMF nhanh hơn và Optimized sử dụng nhiều bộ nhớ hơn.

Thực nghiệm và đánh giá

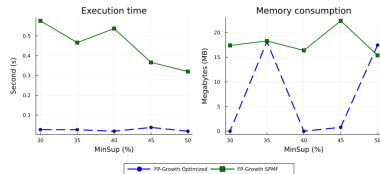
Hiệu năng trên dữ liệu thưa



T10I4D100K



T20I6D100K



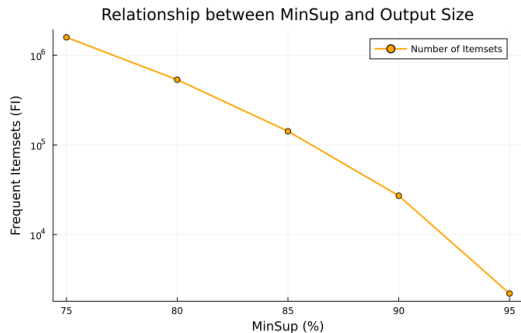
Retail

Mẫu hình chung

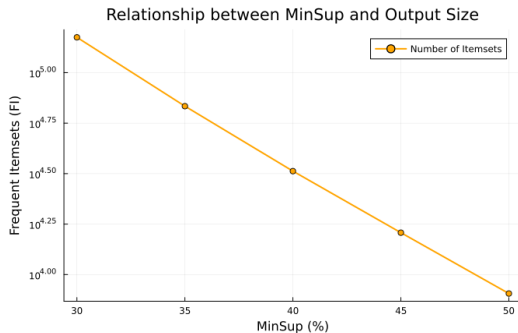
Trên ba tập dữ liệu thưa, Optimized nhanh hơn SPMF tại toàn bộ các ngưỡng được khảo sát. Mức sử dụng bộ nhớ vẫn biến động theo dataset và MinSup nên cần được đọc riêng theo từng cấu hình.

Thực nghiệm và đánh giá

Ảnh hưởng của MinSup trên dữ liệu dày



Connect-4: 2.201 → 1,59 triệu itemsets.



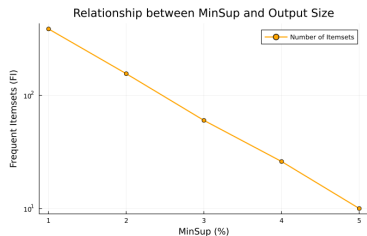
Accidents: 8.057 → 149.545 itemsets.

Thông điệp

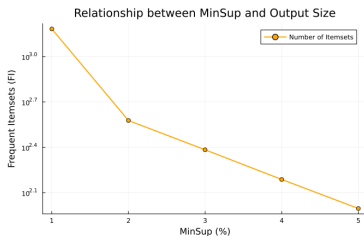
Khi ξ giảm, nhiều tập con vượt ngưỡng hơn. Trên dữ liệu dày, đồng xuất hiện cao làm số frequent itemset tăng rất nhanh.

Thực nghiệm và đánh giá

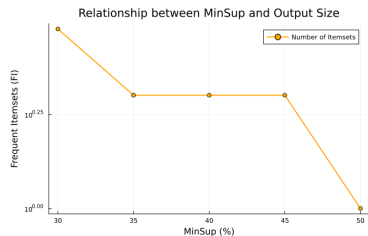
Ảnh hưởng của MinSup trên dữ liệu thưa



T10I4D100K



T20I6D100K



Retail

Giải thích

Không gian item lớn nhưng giao dịch ngắn làm đồng xuất hiện thấp, nên đầu ra tăng chậm hơn dữ liệu dày. Số lượng itemset phụ thuộc vào dữ liệu và MinSup, không phụ thuộc vào phiên bản cài đặt.

Thực nghiệm và đánh giá

Bộ nhớ đỉnh: đánh đổi theo dữ liệu và ngưỡng

Peak memory

Dataset	MinSup	Base	Opt.	Δ
Connect-4	75%	584,10	835,15	+251,05
Connect-4	80%	279,93	239,28	-40,65
T10I4D100K	2%	19,76	2,36	-17,41
T20I6D100K	2%	146,50	113,34	-33,16
Accidents	30%	215,66	268,57	+52,91
Accidents	50%	161,92	161,65	-0,27
Retail	30%	12,43	1,14	-11,28

Đơn vị: MB; Δ = Opt. – Base.

Kết luận bộ nhớ

- Dữ liệu thưa thường giảm RAM rõ rệt.
- Dữ liệu dày, MinSup thấp có thể tăng RAM.
- Connect-4 75% và Accidents 30% là hai ngoại lệ chính.

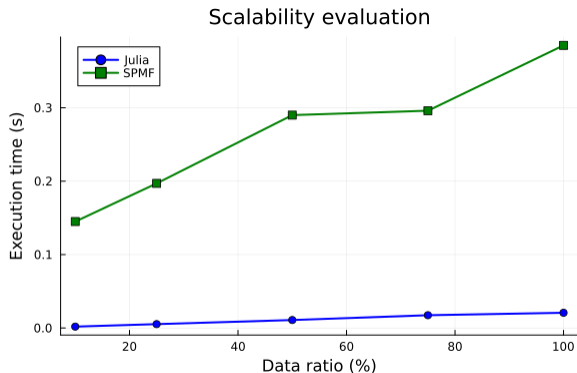
Không tối ưu một chỉ số duy nhất.

Cách đọc kết quả

Đánh giá runtime và peak memory theo từng cấu hình; không kết luận từ một ngưỡng MinSup duy nhất.

Thực nghiệm và đánh giá

Khả năng mở rộng trên Retail



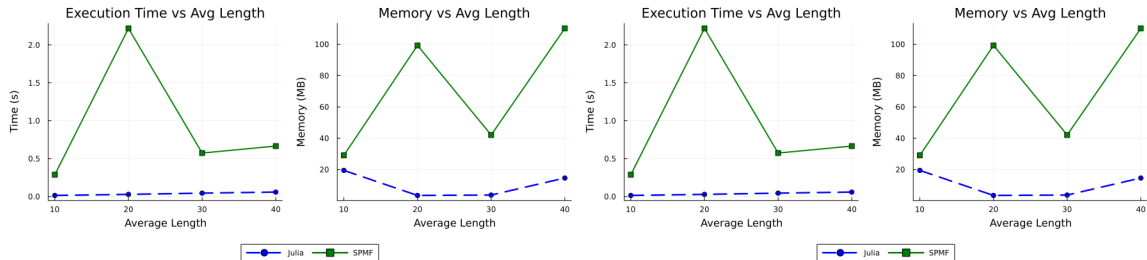
Subset	Giao dịch	Optimized	SPMF
10%	8.817	0,002	0,145
25%	22.041	0,005	0,197
50%	44.081	0,011	0,290
75%	66.122	0,017	0,296
100%	88.162	0,021	0,385

Scaling

Khi số giao dịch tăng 10 lần, thời gian Optimized tăng xấp xỉ 10 lần; kết quả cho thấy xu hướng tăng gần tuyến tính trong cấu hình Retail được khảo sát.

Thực nghiệm và đánh giá

Ảnh hưởng của độ dài giao dịch



Tổng quan theo độ dài giao dịch trung bình.

Thời gian và bộ nhớ khi thay đổi độ dài giao dịch.

Nhận xét

Khi độ dài giao dịch tăng, chi phí chèn vào FP-tree và khai thác conditional tree thường tăng theo. Trong thí nghiệm, thời gian Optimized tăng từ 0,015s lên 0,059s; thời gian SPMF biến động do chỉ được đo một lần. Kết quả này chưa thay thế cho ablation test riêng từng kỹ thuật tối ưu.

Thực nghiệm và đánh giá

Tổng hợp kết quả và hướng tối ưu

Điểm mạnh

- Khớp 100% với SPMF trên 5 benchmark.
- Nhanh hơn SPMF ở đa số cấu hình.
- Gần tuyến tính trên Retail trong cấu hình khảo sát.

Hạn chế

- Dữ liệu dày và MinSup thấp làm output bùng nổ.
- RAM có thể tăng ở một số cấu hình dày.
- Chưa song song hóa các conditional FP-tree độc lập.

Hướng tối ưu

- Song song hóa theo từng item hậu tố.
- Stream output thay vì giữ toàn bộ itemset trong RAM.
- Hỗ trợ closed/maximal itemset để giảm đầu ra.

Thông điệp tổng hợp

Phiên bản tối ưu đạt correctness đầy đủ và thường nhanh hơn, nhưng cần tiếp tục giảm RAM khi dữ liệu dày tạo quá nhiều itemset.

Ứng dụng Market Basket Analysis

Dữ liệu Groceries và tiền xử lý

Thông tin dữ liệu

Thông tin	Giá trị
Nguồn	Groceries từ Kaggle
Số hàng gốc	38.765 hành vi mua hàng
Số khách hàng	3.898
Số sản phẩm	167 loại
Giao dịch sau gom	14.963

Ví dụ dữ liệu gốc

Member	Date	itemDescription
1808	21-07-2015	tropical fruit
2552	05-01-2015	whole milk
2300	19-09-2015	pip fruit

Chiến lược gom giao dịch

Một giao dịch là danh sách mặt hàng sau khi gom theo cặp (Member_number, Date).

Ví dụ sau khi gom giao dịch

Member	Date	itemDescription
1808	21-07-2015	[tropical fruit, rolls/buns, candy]

- Kích thước giỏ hàng trung bình: 2,54 sản phẩm.
- Kích thước tối đa: 10 sản phẩm.
- Dữ liệu rất thưa, cần MinSup thấp để giữ mẫu có ý nghĩa.

Ứng dụng Market Basket Analysis

Frequent itemset và association rule

Cấu hình khai thác

- MinSup = 0,1%, tương đương ít nhất 15 giao dịch.
- Confidence threshold = 5%.
- FP-Growth tối ưu chạy trong 1,085s.

Kết quả frequent itemset

149 itemset kích thước 1, 592 itemset kích thước 2, 9 itemset kích thước 3; tổng cộng 750.

Chỉ số luật kết hợp

$$\text{conf}(X \Rightarrow Y) = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)}$$

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{rsup}(Y)}$$

- Sinh được 450 association rule.
- 94 rule có lift > 1, chiếm 20,89%.
- Lift trung bình toàn bộ rule: 0,8859.

Ứng dụng Market Basket Analysis

10 association rule cao nhất theo lift

#	Rule	Count	Sup%	Conf%	Lift
1	sữa tươi + sữa chua $\Rightarrow$ xúc xích	22	0,147	13,17	2,183
2	xúc xích + sữa tươi $\Rightarrow$ sữa chua	22	0,147	16,42	1,912
3	specialty chocolate $\Rightarrow$ citrus fruit	21	0,140	8,79	1,654
4	xúc xích + sữa chua $\Rightarrow$ sữa tươi	22	0,147	25,58	1,620
5	bột mì $\Rightarrow$ trái cây nhiệt đới	16	0,107	10,96	1,617
6	đồ uống $\Rightarrow$ xúc xích	23	0,154	9,27	1,537
7	nước ngọt có ga + sữa tươi $\Rightarrow$ xúc xích	16	0,107	9,20	1,524
8	khăn giấy $\Rightarrow$ bánh ngọt	26	0,174	7,85	1,519
9	phô mai chế biến $\Rightarrow$ rau củ dạng rễ	16	0,107	10,53	1,513
10	phô mai cứng $\Rightarrow$ trái cây có hạt	16	0,107	7,27	1,483

Nhận xét

Các rule lift cao thường có support thấp nhưng thể hiện tương quan dương đáng chú ý trong dữ liệu giỏ hàng thưa.

Ứng dụng Market Basket Analysis

Nhóm 1: Sữa tươi/sữa chua và xúc xích

Phân tích luật 1, 2, 4

- Luật 1 có lift 2,183: khi khách đã mua sữa tươi và sữa chua, khả năng mua thêm xúc xích tăng 2,183 lần so với bình thường.
- Luật 2 có lift 1,912: xúc xích và sữa tươi thường kéo theo nhu cầu mua sữa chua.
- Luật 4 có confidence 25,58%, cao nhất bảng: khoảng 1 trong 4 khách mua xúc xích + sữa chua sẽ mua thêm sữa tươi.

Ý nghĩa và hành động

- Đây là cụm sản phẩm cho bữa sáng hoặc bữa ăn nhẹ nhanh, phù hợp với gia đình trẻ, học sinh, trẻ em.
- Đặt xúc xích ăn liền/xúc xích tươi cạnh tủ mát sữa tươi và sữa chua.
- Tạo combo “Bữa sáng dinh dưỡng” gồm sữa tươi, sữa chua và xúc xích với ưu đãi nhẹ.

Phân tích luật 3, 6, 7, 8

- Luật 6, 7: khách mua đồ uống hoặc nước ngọt có ga + sữa tươi có xu hướng mua kèm xúc xích như đồ ăn mặn/đồ nhấm.
- Luật 3 có lift 1,654: specialty chocolate thường đi kèm citrus fruit, phù hợp nhu cầu ăn vặt hoặc quà tặng nhỏ.
- Luật 8: khăn giấy $\Rightarrow$ bánh ngọt, gợi ý bối cảnh dùng bánh tươi/ăn nhẹ cần vật dụng đi kèm.

Ý nghĩa và hành động

- Nhóm này phản ánh nhu cầu ăn vặt, giải trí hoặc tiệc nhỏ.
- Thiết kế kệ “Tiệc ngọt” đặt socola cạnh quầy trái cây tươi.
- Đặt quầy bánh ngọt/bánh tươi gần khăn giấy, màng bọc thực phẩm; đặt xúc xích gần khu đồ uống để tăng cross-selling.

Ứng dụng Market Basket Analysis

Nhóm 3: Nguyên liệu nấu ăn và chuẩn bị bữa chính

Phân tích luật 5, 9, 10

- Luật 5 có lift 1,617: bột mì đi kèm trái cây nhiệt đới, có thể gắn với nhu cầu làm bánh trái cây.
- Luật 9: phô mai chế biến $\Rightarrow$ rau củ dạng rế, phù hợp các món bỏ lò, hầm hoặc nấu kiểu Tây.
- Luật 10: phô mai cứng $\Rightarrow$ trái cây có hạt, gợi ý các món salad, khai vị hoặc platter.

Ý nghĩa và hành động

- Đối tượng chính là khách thích tự nấu ăn hoặc làm bánh tại nhà.
- Đặt bảng gợi ý công thức: “Bánh tart trái cây nhiệt đới”, “Salad phô mai”, “Rau củ bỏ lò phô mai”.
- Sắp xếp phô mai gần khu rau củ Đà Lạt/rau củ ôn đới để khách dễ hoàn thiện giỏ hàng.

Insight chính

- Lift cao là tín hiệu gợi ý, nhưng cần xem kèm support để tránh kết luận quá mức.
- Nhóm sữa tươi, sữa chua và xúc xích thể hiện xu hướng đồng mua nổi bật.
- Kết quả phù hợp để đề xuất combo, trưng bày gần nhau và kiểm chứng bằng thử nghiệm thực tế.

Hạn chế

- Top rule chỉ có 16–26 giao dịch hỗ trợ, dễ nhạy với nhiễu.
- Mới chạy một cấu hình: MinSup 0,1%, confidence 5%.
- Lift trung bình 0,8859 cho thấy đa số rule không tương quan dương.

- ① FP-Growth khai thác frequent itemset hiệu quả nhờ nén dữ liệu bằng FP-tree và loại bỏ bước sinh candidate.
- ② Cài đặt Julia đạt match rate 100% với SPMF trên toy datasets và năm tập benchmark.
- ③ Phiên bản tối ưu nhanh hơn SPMF trong phần lớn cấu hình, đặc biệt trên dữ liệu thưa; trên dữ liệu dày với đầu ra rất lớn cần chú ý bộ nhớ.
- ④ Ứng dụng Groceries tạo 750 frequent itemset và 450 rule, cung cấp gợi ý trưng bày và khuyến mãi chéo có thể kiểm thử thực tế.

-  J. Han, J. Pei, and Y. Yin. Mining frequent patterns without candidate generation. SIGMOD, 2000.
-  J. Han, J. Pei, Y. Yin, and R. Mao. Mining frequent patterns without candidate generation: A frequent-pattern tree approach. DMKD, 2004.
-  R. Agrawal, T. Imielinski, and A. Swami. Mining association rules between sets of items in large databases. SIGMOD, 1993.
-  P. Fournier-Viger et al. The SPMF open-source data mining library. JMLR, 2014.
-  UCI Machine Learning Repository and Kaggle Groceries Dataset.

Cảm ơn thầy/cô và các bạn đã lắng nghe!